Invoice automatic scanning and text predictions

Solution methodology

BUSINESS OVERVIEW:

People have been relying on paper invoices for a very long time but these days, all have become digital and so are invoices. Reconciling digital invoices is currently a man-made job but this can be automated to avoid men spending hours on browsing through several invoices and noting things down in a ledger. This project deals with detecting three important classes from the invoices. They are:

1. Invoice number
2. Billing Date
3. Total amount

APPLICATIONS:

This project can be applicable to any field that is currently going through all the bills manually to jot it down in a ledger.

TECH STACK:

Language: Python

Object detection : YOLO V4

Text Recognition : Tesseract OCR

Environment: Google Colab

APPROACH:

1. **Setting up and Installation to run Yolov4**

   Downloading AlexeyAB's famous repository and we will be adjusting the Makefile to enable OPENCV and GPU for darknet and then build darknet.

2. **Downloading pre-trained YOLOv4 weights**

   YOLOv4 has been trained already on the coco dataset which has 80 classes that it can predict. We will take these pretrained weights to see how it gives result on some of the images.

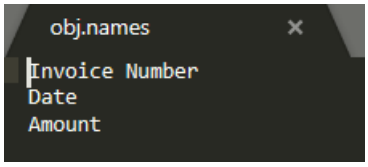
3. **Creating display functions to display the predicted class**
4. **Data collection and Labeling with LabelImg**

To create a custom object detector, we need a good dataset of images and labels for better training of the model so we will label our images using annotation tools LabelImg

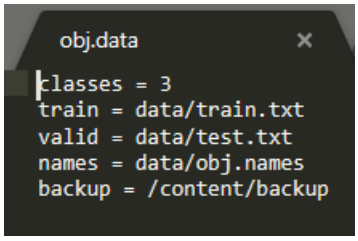
5. **Configuring Files for Training**

   This step involves configuring custom .cfg, obj.data, obj.names, train.txt and test.txt files.

   a. Configuring all the needed variables based on class in the config file
   b. Creating obj.names and obj.data files
      i. obj.names : Classes to be detected



      ii. obj.data:



   c. Configuring train.txt and test.txt
6. **Download pre-trained weights for the convolutional layers**
7. **Training Custom Object Detector**
8. **Evaluating the model using Mean Average precision**
9. **Predict image classes and save the co-ordinates separately**
10. **Detecting text from the predicted class**
    a. Importing pytesseract and setting environment variable for english trained data
    b. Getting list of predicted files from the directory
    c. Using tesseract pretrained LSTM model to extract the text
    d. Fine tuning the LSTM model.

PROJECT TAKEAWAYS:

1. Setting up YOLO V4
2. Data labelling with LabelImg
3. Understanding YOLO architecture
4. Understanding how to use pre trained models of YOLO
5. Configuring YOLO to any project
6. Understanding LSTM
7. Understanding Object detection

8. Training Custom Object Detector with YOLO
9. Understanding Text extraction using Tesseract
10. Understanding what is OCR
11. Using tesseract pretained LSTM
12. Configuring Tessaract model to our needs
13. Fine tuning Tessaract model
14. Understanding Mean Average Precision
15. Storing and displaying detected classes in an image.